Préparé par

Samir Ouhmmou et Taoufik Mansouri

Jenkins

Lab complet : Pipeline CI/CD
et Jenkinsfile expliqué en détail



Installation • Configuration • Jenkinsfile • Labs pas à pas
Commandes Linux / Docker / Git / Maven / Jenkins CLI • Dépannage

23 octobre 2026

Table des matières

Chapitre 1

Introduction à Jenkins

1.1 Qu'est-ce que Jenkins ?

Jenkins est un serveur d'automatisation open source écrit en Java. Il sert à automatiser les tâches répétitives du cycle de vie d'un logiciel : compiler le code, lancer les tests, construire des images Docker, déployer sur un serveur, envoyer des notifications, etc.

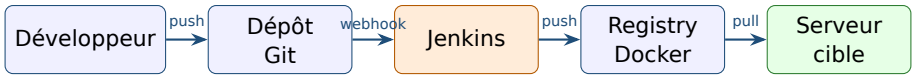
Son principal atout est son écosystème : plus de 1800 plugins permettent de l'intégrer avec Git, GitHub, GitLab, Docker, Kubernetes, SonarQube, Slack, AWS, Ansible, et bien d'autres outils.

1.2 CI/CD : les notions de base

CI / CD

Intégration continue (CI) : à chaque modification du code (*commit*, *push*), le projet est automatiquement compilé et testé pour détecter les erreurs au plus tôt.
Livraison continue (Continuous Delivery) : l'application validée est automatiquement préparée pour la mise en production (artefact, image Docker), la mise en production restant déclenchée manuellement.
Déploiement continu (Continuous Deployment) : la mise en production est elle aussi entièrement automatique si tous les tests passent.

Un flux CI/CD typique ressemble à ceci :



1.3 Architecture de Jenkins

Jenkins fonctionne avec une architecture **contrôleur / agents** :

- Le **contrôleur** (anciennement *master*) héberge l'interface web, la configuration, l'ordonnancement des jobs et l'historique. Il doit idéalement ne pas exécuter lui-même les builds lourds.
- Les **agents** (anciennement *slaves*, aussi appelés *nœuds*) exécutent réellement les builds. Ce peut être une machine physique, une VM ou un conteneur Docker éphémère.
- Les **exécuteurs** (*executors*) sont les « emplacements » de build disponibles sur un nœud : un nœud avec 2 exécuteurs peut faire tourner 2 builds en parallèle.

Le répertoire de données de Jenkins s'appelle `JENKINS_HOME` (par défaut `/var/lib/jenkins` sous Linux avec le paquet `.deb`, ou `/var/jenkins_home` dans l'image Docker). Il contient les jobs, les plugins, les credentials chiffrés et les espaces de travail.

1.4 Vocabulaire indispensable

Terme	Signification
Job / Projet	Une tâche configurée dans Jenkins (Freestyle, Pipeline, Multibranch, etc.).
Build / Run	Une exécution d'un job. Chaque build a un numéro (`#1`, `#2`, …).
Pipeline	Une suite d'étapes automatisées décrite sous forme de code (le *Jenkinsfile*).
Jenkinsfile	Fichier texte (syntaxe Groovy) versionné avec le code source, qui décrit le pipeline.
Stage	Une grande phase logique du pipeline (Build, Test, Deploy…) visible dans l'interface.
Step	Une action élémentaire dans un stage (`sh`, `echo`, `junit`…).
Agent / Node	Machine ou conteneur qui exécute le pipeline.
Workspace	Dossier de travail sur l'agent où le code est récupéré et construit.
Artefact	Fichier produit par un build (JAR, ZIP, rapport) et archivé par Jenkins.
Credential	Secret stocké de façon chiffrée (mot de passe, token, clé SSH).
Plugin	Extension qui ajoute des fonctionnalités à Jenkins.
Webhook	Appel HTTP envoyé par GitHub/GitLab à Jenkins pour déclencher un build dès un push.
Trigger	Mécanisme de déclenchement (cron, polling SCM, webhook, autre job).

Chapitre 2

Objectifs, prérequis et organisation du lab

Objectifs pédagogiques

À la fin de ce lab, vous saurez :

- installer et configurer Jenkins (Docker ou Ubuntu) ;
- écrire et comprendre chaque partie d'un `Jenkinsfile` déclaratif ;
- construire un pipeline complet : checkout, build Maven, tests, archivage, image Docker, déploiement ;
- gérer les credentials de façon sécurisée ;
- utiliser les paramètres, conditions, parallélisme, approbations manuelles et post-actions ;
- déclencher les builds automatiquement (webhook GitHub, cron) ;
- diagnostiquer les erreurs les plus courantes.

2.1 Prérequis

- Une machine Linux (Ubuntu 22.04/24.04 recommandé), ou Windows/macOS avec Docker Desktop, avec au moins 2 Go de RAM libres et 10 Go de disque.
- Docker installé (`docker --version`).
- Git et un compte GitHub (ou GitLab).
- Notions de base du terminal Linux.
- Un petit projet Java/Maven d'exemple (fourni ci-dessous), mais le lab se transpose facilement à Node.js, Python, etc.

2.2 Projet fil rouge

Tout au long du lab, nous utiliserons un projet Java minimal nommé `demo-app` avec la structure suivante :

```
demo-app/
|-- Jenkinsfile
|-- Dockerfile
|-- pom.xml
`-- src/
    |-- main/java/com/example/App.java
    `-- test/java/com/example/AppTest.java
```

Listing 2.1: Structure du dépôt Git

Vous pouvez générer un projet Maven de départ avec :

```
mvn archetype:generate -DgroupId=com.example -DartifactId=demo-app \
    -DarchetypeArtifactId=maven-archetype-quickstart \
    -DarchetypeVersion=1.4 -DinteractiveMode=false
cd demo-app
git init
git add .
git commit -m "Initial commit"
```

Listing 2.2: Génération du projet Maven

- `mvn archetype:generate` : génère un squelette de projet à partir d'un modèle (archetype).
- `-DgroupId`, `-DartifactId` : identifiants Maven du projet (organisation et nom du module).
- `-DinteractiveMode=false` : n'attend aucune saisie, utilise les valeurs données.
- `git init` : initialise un dépôt Git local ; `git add .` met tous les fichiers en zone d'index ; `git commit -m` enregistre le premier commit.

Chapitre 3

Lab 0 – Installation et configuration de Jenkins

Deux méthodes sont présentées. Choisissez celle qui vous convient : la méthode **Docker** est la plus rapide pour un lab, la méthode **paquet Ubuntu** est la plus proche d'une installation de serveur.

3.1 Méthode A : installation avec Docker

3.1.1 Lancer le conteneur Jenkins

```
docker run -d --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --restart unless-stopped \
  jenkins/jenkins:lts-jdk17
```

Listing 3.1: Démarrage de Jenkins dans Docker

Explication option par option :

Option	Rôle
`docker run -d`	Crée et démarre un conteneur en arrière-plan (*detached*).
`--name jenkins`	Donne le nom `jenkins` au conteneur pour le retrouver facilement.
`-p 8080:8080`	Publie le port web : l'interface sera sur `http://localhost:8080`.
`-p 50000:50000`	Port utilisé par les agents distants pour se connecter au contrôleur (JNLP/WebSocket).
`-v jenkins_home:/var/jenkins_home`	Volume persistant : conserve jobs, plugins et config même si le conteneur est supprimé.
`-v /var/run/docker.sock:...`	Donne au conteneur accès au démon Docker de l'hôte (nécessaire pour construire des images depuis les pipelines).
`--restart unless-stopped`	Redémarre automatiquement le conteneur après un reboot, sauf arrêt manuel.
`jenkins/jenkins:lts-jdk17`	Image officielle, version LTS (support long terme) avec Java 17.

Sécurité du socket Docker

Monter `/var/run/docker.sock` donne à Jenkins un contrôle quasi total sur l'hôte (équivalent root). C'est acceptable pour un lab, mais à éviter tel quel en production : préférez des agents

dédiés ou Docker-in-Docker avec TLS.

3.1.2 Image personnalisée avec le client Docker

L'image officielle ne contient pas la commande `docker`. Pour l'utiliser dans les pipelines, créez une image dérivée :

```
FROM jenkins/jenkins:lts-jdk17
USER root
RUN apt-get update && apt-get install -y docker.io maven git \
    && rm -rf /var/lib/apt/lists/*
RUN groupadd -f docker && usermod -aG docker jenkins
USER jenkins
```

Listing 3.2: Dockerfile de l'image Jenkins personnalisée

- `FROM` : image de base.
- `USER root` : passe en root pour pouvoir installer des paquets.
- `apt-get install -y docker.io maven git` : installe le client Docker, Maven et Git dans l'image.
- `usermod -aG docker jenkins` : ajoute l'utilisateur `jenkins` au groupe `docker`.
- `USER jenkins` : revient à l'utilisateur non privilégié, comme dans l'image officielle.

```
docker build -t jenkins-custom .
docker rm -f jenkins
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add $(stat -c '%g' /var/run/docker.sock) \
  jenkins-custom
```

Listing 3.3: Construction et lancement de l'image personnalisée

`--group-add` ajoute le groupe propriétaire du socket Docker de l'hôte au processus, ce qui évite l'erreur « permission denied » sur `/var/run/docker.sock`.

3.2 Méthode B : installation sur Ubuntu

```
# 1. Installer Java 17
sudo apt update
sudo apt install -y fontconfig openjdk-17-jre
java -version

# 2. Ajouter la cle GPG et le depot Jenkins (LTS)
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

# 3. Installer Jenkins
sudo apt update
sudo apt install -y jenkins
```

```
# 4. Demarrer et activer au boot
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

Listing 3.4: Installation de Jenkins avec apt

- `apt install openjdk-17-jre` : Jenkins a besoin d'un JRE ; `fontconfig` est requis pour le rendu des graphiques.
- `wget -O ... .key` : télécharge la clé GPG qui signe les paquets, afin qu'`apt` vérifie leur authenticité.
- La ligne `echo "deb [signed-by=...] ..."` déclare le dépôt officiel Jenkins dans `/etc/apt/sources.list.d/`.
- `systemctl enable --now jenkins` : active le démarrage automatique *et* lance le service immédiatement.
- `systemctl status jenkins` : vérifie que le service est `active (running)`.

Astuce

Les URL et le nom de la clé GPG peuvent évoluer d'une année à l'autre. En cas d'erreur de signature, consultez la page officielle d'installation sur `jenkins.io` pour obtenir la clé à jour.

Pour l'ouverture du pare-feu (si `ufw` est actif) :

```
sudo ufw allow 8080/tcp
sudo ufw status
```

3.3 Assistant de premier démarrage

1. Ouvrez `http://localhost:8080` (ou `http://IP_DU_SERVEUR:8080`).
2. Jenkins demande le mot de passe administrateur initial. Récupérez-le :

```
# Docker
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
# Paquet Ubuntu
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

3. Choisissez **Install suggested plugins** (Git, Pipeline, Credentials, etc.).
4. Créez le premier utilisateur administrateur.
5. Confirmez l'URL de Jenkins (*Jenkins URL*), puis cliquez sur **Start using Jenkins**.

3.4 Plugins à installer pour ce lab

Menu : **Manage Jenkins** → **Plugins** → **Available plugins**.

Plugin	Utilité
`Pipeline`	Support des pipelines (Jenkinsfile), généralement déjà installé.
`Git`	Récupération du code depuis Git (`checkout`, `git`).
`GitHub Integration`	Webhooks GitHub, déclencheur `githubPush`.
`Docker Pipeline`	Syntaxe `agent { docker { ... } }` et objet `docker`.
`Credentials Binding`	Étape `withCredentials`, injection de secrets.
`SSH Agent`	Étape `sshagent` pour les connexions SSH.
`JUnit`	Publication des résultats de tests (`junit`).
`Timestamper`	Option `timestamps()` dans les logs.
`Workspace Cleanup`	Étape `cleanWs()`.
`Blue Ocean (optionnel)`	Visualisation moderne des pipelines.

3.5 Configuration des outils

Menu : **Manage Jenkins** → **Tools**. C'est ici que l'on déclare les noms utilisés dans la directive `tools` du Jenkinsfile.

- **JDK** : ajoutez une installation nommée `JDK-17` (cochez *Install automatically* ou indiquez `JAVA_HOME`).
- **Maven** : ajoutez une installation nommée `Maven-3.9` (version 3.9.x, installation automatique depuis Apache).

Attention

Le nom saisi (`Maven-3.9`, `JDK-17`) doit être **identique** à celui utilisé dans le Jenkinsfile, majuscules comprises. Sinon : l'erreur `Tool type "maven" does not have an install of "..." configured`.

3.6 Création des credentials

Menu : **Manage Jenkins** → **Credentials** → **System** → **Global credentials** → **Add Credentials**.

ID	Type	Contenu
`dockerhub-creds`	Username with password	Identifiant Docker Hub + mot de passe ou token d'accès.
`github-creds`	Username with password	Utilisateur GitHub + Personal Access Token (pour dépôt privé).
`deploy-ssh-key`	SSH Username with private key	Utilisateur du serveur cible + clé privée.
`slack-token`	Secret text	Jeton d'API d'un service externe.

Le champ **ID** est la valeur référencée dans le Jenkinsfile : choisissez-le soigneusement, il ne peut plus être renommé simplement ensuite.

Chapitre 4

Les pipelines : concepts

4.1 Pipeline déclaratif vs scripté

Jenkins propose deux syntaxes, toutes deux basées sur Groovy :

	Déclaratif	Scripté
Racine	`pipeline { ... }`	`node { ... }`
Structure	Rigide, sections prédéfinies	Libre, code Groovy complet
Lisibilité	Excellente, recommandée	Plus technique
Validation	Vérifiée avant exécution	Erreurs détectées à l'exécution
Gestion d'erreurs	Bloc `post`	`try/catch/finally`
Usage conseillé	95 % des cas	Logique très dynamique

Ce lab utilise principalement la syntaxe **déclarative**.

4.2 Où placer le Jenkinsfile ?

Deux possibilités :

1. **Pipeline script** : le code est écrit directement dans l'interface Jenkins. Pratique pour tester, mais non versionné.
2. **Pipeline script from SCM** : Jenkins lit le fichier `Jenkinsfile` à la racine du dépôt Git. C'est la bonne pratique (*Pipeline as Code*) : le pipeline évolue avec le code, est relu en revue, et historisé.

4.3 Créer un job Pipeline

1. Tableau de bord → **New Item**.
2. Saisissez un nom (ex. `demo-app-pipeline`), choisissez **Pipeline**, validez avec **OK**.
3. Section **Pipeline** → *Definition* : **Pipeline script from SCM**.
4. *SCM* : **Git**, indiquez l'URL du dépôt et les credentials si le dépôt est privé.
5. *Branch Specifier* : `*/main` (ou `*/master`).
6. *Script Path* : `Jenkinsfile`.
7. **Save**, puis **Build Now**.

4.4 Cycle d'exécution d'un build

1. Jenkins reçoit un déclencheur (clic, webhook, cron, polling).
2. Le build est placé dans la file d'attente (*Build Queue*) jusqu'à ce qu'un exécuteur soit libre.
3. L'agent crée (ou réutilise) un **workspace**.
4. Le Jenkinsfile est interprété : chaque `stage` est exécuté dans l'ordre.
5. Si un step échoue, les stages suivants sont ignorés et le bloc `post` s'exécute.
6. Le résultat final est `SUCCESS`, `UNSTABLE`, `FAILURE` ou `ABORTED`.

Résultat	**Signification**
`SUCCESS`	Toutes les étapes se sont bien déroulées (pastille bleue/verte).
`UNSTABLE`	Le build s'est terminé mais avec des tests en échec ou des avertissements (jaune).
`FAILURE`	Une étape a échoué (rouge).
`ABORTED`	Le build a été interrompu manuellement ou par un timeout (gris).
`NOT_BUILT`	Étape non exécutée (ex. condition `when` fausse).

Chapitre 5

Anatomie détaillée d’un Jenkinsfile

Voici un Jenkinsfile déclaratif complet. Nous allons ensuite le décortiquer directive par directive.

```
pipeline {
    agent any

    tools {
        maven 'Maven-3.9'
        jdk   'JDK-17'
    }

    environment {
        APP_NAME = 'demo-app'
        VERSION  = "1.0.${BUILD_NUMBER}"
    }

    options {
        timestamps()
        timeout(time: 15, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10'))
        disableConcurrentBuilds()
    }

    parameters {
        string(name: 'BRANCH', defaultValue: 'main', description: 'Branche a construire')
        booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Executer les tests ?')
    }

    triggers {
        pollSCM('H/5 * * * *')
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
                sh 'git log -1 --oneline'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn -B clean compile'
            }
        }

        stage('Test') {
            when { expression { params.RUN_TESTS } }
            steps {
                sh 'mvn -B test'
            }
            post {
                always {
                    junit 'target/surefire-reports/*.xml'
                }
```

```
            }
        }

        stage('Package') {
            steps {
                sh 'mvn -B package -DskipTests'
                archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            }
        }
    }

    post {
        success { echo "Build ${VERSION} reussi" }
        failure { echo 'Build en echec' }
        always  { cleanWs() }
    }
}
```

Listing 5.1: Jenkinsfile de référence (pipeline CI complet)

5.1 `pipeline { }` : le bloc racine

Tout pipeline déclaratif est entièrement contenu dans un bloc `pipeline`. Rien ne peut exister en dehors (sauf des `import` ou fonctions Groovy en tête de fichier). Ce bloc ne peut contenir que des *sections* (`agent`, `stages`, `post`…) et des *directives* (`environment`, `options`…).

5.2 `agent` : où s'exécute le pipeline

La directive `agent` est **obligatoire**. Elle indique sur quelle machine (ou dans quel conteneur) le code s'exécute.

```
agent any                          // n'importe quel agent disponible

agent none                         // aucun agent global : chaque stage doit definir le sien

agent { label 'linux && docker' }  // un agent possedant ces labels

agent {
    docker {                       // execute dans un conteneur
        image 'maven:3.9-eclipse-temurin-17'
        args  '-v $HOME/.m2:/root/.m2'
    }
}

agent {
    dockerfile {                   // construit puis utilise l'image d'un Dockerfile du depot
        filename 'Dockerfile.build'
    }
}
```

Listing 5.2: Les différentes formes de la directive agent

- `agent any` : le plus simple, convient pour un lab.
- `agent none` : utile quand les stages ont des besoins différents ; on déclare alors `agent` au niveau de chaque `stage`.

- `label` : sélectionne un nœud via ses étiquettes (définies dans **Manage Jenkins** → **Nodes**).
- `docker` : Jenkins démarre un conteneur à partir de l'image, y exécute les steps, puis le supprime. Les outils (Maven, Node, Python) n'ont alors pas besoin d'être installés sur l'agent.

5.3 `tools` : outils préconfigurés

```
tools {
    maven 'Maven-3.9'
    jdk   'JDK-17'
}
```

Ajoute automatiquement Maven et le JDK au `PATH` pour la durée du build. Les noms doivent correspondre à ceux déclarés dans **Manage Jenkins** → **Tools**. Types supportés : `maven`, `jdk`, `gradle`, `nodejs` (avec plugin), `git`.

5.4 `environment` : variables d'environnement

```
environment {
    APP_NAME = 'demo-app'
    VERSION  = "1.0.${BUILD_NUMBER}"
    DOCKER_CREDS = credentials('dockerhub-creds')
}
```

- Définit des variables disponibles dans tous les stages (ou uniquement dans un stage si placée dedans).
- Les **guillemets simples** `'...'` produisent du texte littéral ; les **guillemets doubles** `"..."` permettent l'interpolation Groovy de `${VAR}`.
- `credentials('id')` récupère un credential. Pour un type *Username/Password*, Jenkins crée automatiquement trois variables : `DOCKER_CREDS` (au format `user:password`), `DOCKER_CREDS_USR` et `DOCKER_CREDS_PSW`. Leur valeur est masquée (****) dans les logs.

5.5 `options` : réglages du pipeline

Option	Effet
`timestamps()`	Ajoute l'heure devant chaque ligne du log.
`timeout(time: 15, unit: 'MINUTES')`	Interrompt le build s'il dépasse 15 minutes.
`buildDiscarder(logRotator(numToKeepStr: '10'))`	Ne conserve que les 10 derniers builds (économie d'espace disque).
`disableConcurrentBuilds()`	Interdit deux builds simultanés du même job.
`skipDefaultCheckout()`	Désactive le checkout automatique avant le premier stage.
`retry(3)`	Relance tout le pipeline jusqu'à 3 fois en cas d'échec.

Option	Effet
`ansiColor('xterm')`	Affiche les couleurs ANSI (plugin AnsiColor).

5.6 `parameters` : paramètres du build

Les paramètres rendent le job « paramétrable » : le bouton **Build Now** devient **Build with Parameters**.

```
parameters {
    string(name: 'BRANCH', defaultValue: 'main', description: 'Branche')
    booleanParam(name: 'RUN_TESTS', defaultValue: true, description: 'Lancer les tests')
    choice(name: 'ENV', choices: ['dev', 'staging', 'prod'], description: 'Environnement cible')
    password(name: 'TOKEN', defaultValue: '', description: 'Jeton temporaire')
    text(name: 'NOTES', defaultValue: '', description: 'Notes de release')
}
```

On y accède via l'objet `params` : `params.BRANCH`, `params.RUN_TESTS`. Dans un script shell, ils sont aussi exposés comme variables d'environnement (`$BRANCH`).

Attention

Au tout premier build après l'ajout d'un bloc `parameters`, Jenkins ne connaît pas encore les paramètres : le premier run utilise les valeurs par défaut. Les builds suivants affichent bien le formulaire.

5.7 `triggers` : déclenchement automatique

```
triggers {
    cron('H 2 * * *')            // chaque nuit vers 2h (le H repartit la charge)
    pollSCM('H/5 * * * *')       // interroge Git toutes les ~5 minutes
    // githubPush()              // declenche par webhook GitHub (plus efficace)
    // upstream(upstreamProjects: 'autre-job', threshold: hudson.model.Result.SUCCESS)
}
```

- `cron` : exécution planifiée, indépendante des changements de code.
- `pollSCM` : Jenkins interroge périodiquement Git et ne construit que s'il y a un nouveau commit. Simple, mais consomme des ressources.
- `githubPush` : déclenche immédiatement à la réception d'un webhook (voir chapitre **??**).

5.8 `stages` et `stage` : les phases

`stages` contient une ou plusieurs directives `stage`. Chaque `stage` **doit** avoir un nom unique et un bloc `steps` (ou `parallel`/`stages` imbriqués). Les stages s'exécutent séquentiellement ; un échec interrompt la suite.

Le nom du stage apparaît dans la *Stage View* : choisissez des noms courts et clairs.

5.9 `steps` : les actions

C'est dans `steps` que l'on écrit ce qui s'exécute réellement. Les steps les plus utilisés :

Step	Rôle
`echo 'message'`	Affiche un message dans le log.
`sh 'commande'`	Exécute une commande shell (Linux/macOS). Échoue si le code retour est différent de 0.
`bat 'commande'`	Équivalent de `sh` sous Windows.
`checkout scm`	Récupère le code du dépôt configuré dans le job (SCM courant).
`git url: '...', branch: '...'`	Clone un dépôt explicitement.
`dir('sous-dossier') { ... }`	Exécute les steps dans un sous-répertoire.
`withCredentials([...]) { ... }`	Injecte temporairement des secrets.
`sshagent(['id']) { ... }`	Charge une clé SSH le temps du bloc.
`junit 'chemin/*.xml'`	Publie les résultats de tests JUnit.
`archiveArtifacts artifacts: '...'`	Conserve des fichiers produits par le build.
`stash / unstash`	Transmet des fichiers entre stages ou agents.
`timeout(time: 5, unit: 'MINUTES') { }`	Limite la durée d'un bloc.
`retry(3) { ... }`	Réessaie un bloc jusqu'à 3 fois.
`sleep time: 10, unit: 'SECONDS'`	Pause.
`input message: '...'`	Attend une validation manuelle.
`script { ... }`	Ouvre un bloc Groovy libre dans un pipeline déclaratif.
`error 'message'`	Fait échouer volontairement le build.
`cleanWs()`	Supprime le contenu du workspace.
`build job: 'autre-job'`	Déclenche un autre job.

5.9.1 Le step `sh` en détail

```groovy
sh 'mvn -B clean package'                       // une commande

sh '''
    echo "Debut"
    mvn -B test
    echo "Fin"
'''                                              // plusieurs commandes (triple quotes)

def sortie = sh(script: 'git rev-parse --short HEAD', returnStdout: true).trim()
def code   = sh(script: 'ls /inexistant', returnStatus: true)   // ne fait pas echouer
```

- Par défaut, `sh` exécute avec `/bin/sh -xe` : chaque commande est affichée (`-x`) et le script s'arrête à la première erreur (`-e`).
- `returnStdout: true` renvoie la sortie standard sous forme de chaîne (pensez à `.trim()`).
- `returnStatus: true` renvoie le code de retour au lieu de faire échouer le build.

- Les **triples guillemets simples** `'''` évitent l'interpolation Groovy et sont recommandés pour les scripts contenant des secrets.

5.10 `when` : exécuter un stage sous condition

```groovy
stage('Deploy to prod') {
    when {
        branch 'main'                          // seulement sur la branche main
    }
    steps { echo 'Deploiement en production' }
}

stage('Condition combinee') {
    when {
        allOf {
            branch 'main'
            expression { params.ENV == 'prod' }
            not { changeRequest() }
        }
    }
    steps { echo 'OK' }
}
```

Conditions disponibles : `branch`, `environment`, `expression`, `changeRequest`, `tag`, `changeset`, `allOf`, `anyOf`, `not`. Si la condition est fausse, le stage est simplement *sauté* (affiché en gris).

5.11 `post` : actions de fin

Le bloc `post` s'exécute *après* les stages, selon le résultat. Il peut se placer au niveau du pipeline ou d'un stage.

Condition	S'exécute…
`always`	Toujours, quel que soit le résultat.
`success`	Uniquement si le build réussit.
`failure`	Uniquement si le build échoue.
`unstable`	Si le build est instable (tests en échec par exemple).
`changed`	Si le résultat diffère de celui du build précédent.
`fixed`	Si le build réussit alors que le précédent avait échoué.
`regression`	Si le build est pire que le précédent.
`aborted`	Si le build est interrompu manuellement.
`cleanup`	Toujours, *en dernier*, après tous les autres blocs `post`.

Astuce

Utilisez `post { always { cleanWs() } }` pour libérer l'espace disque, et `post { failure { ... } }` pour envoyer un mail ou un message Slack.